

Software Size Measurement

A decorative graphic consisting of a 3x3 grid of squares. The center square is dark blue and contains the white number '13'. The surrounding squares are a lighter shade of blue.

13

Lesson

Outline

1. Software Size Measurement

13.1 Software Size Measurement

■ Basic steps for software project estimation

1. Estimate the size of the development product.
2. Estimate the effort in person-months or person-hours.
3. Estimate the schedule in calendar months.
4. Estimate the project cost.

The main methods of software size measurement are LOC and FP.

■ LOC (Lines of Code)

Simplest and most widely used metric. In this method, comments and blank lines should not be counted.

■ Disadvantages of Using LOC

- Size can vary with coding style.
- Focuses on coding activity alone.
- Correlates poorly with quality and efficiency of code.
- Higher level programming languages, code reuse, etc

13.1 Software Size Measurement

■ Methods of software size estimation

There are 4 methods can be used to estimate the size of software.

- Comparison method
- Analogy method
- Work breakdown method
- Equation method

The first two methods are mainly used for software size estimation in the early stages of a project, such as project opportunity research, feasibility study, and cost estimation for project initiation.

The third method, work breakdown method used to measure the software size of software components, interfaces, and even the smallest program unit.

The equation method is the most widely used method for software size metrics in the software industry, which can meet the needs of software size measurement throughout the entire software lifecycle.

13.1 Software Size Measurement

■ Software size estimation approach: Comparison(类比法)

Essentially, comparison method involves finding a group of completed projects with project attributes similar to those of the proposed project, and using the data from those completed projects to provide a guide for the estimate of the effort and duration for your new project.

Comparison method is useful when enough project attributes and a range for the functional size are known. This allows the estimator to adequately gauge(测量) that the comparison projects are similar.

This method is suitable for cost estimation when requirements are vague or uncertain.

13.1 Software Size Measurement

■ Software size estimation approach: Comparison(类比法)

Comparison belongs to the method of "**calculation**" as the main method. When the project to be evaluated is similar to the completed project in certain attributes, such as application domain, system size, complexity, development team experience etc., comparison method can be used. It is based on a large amount of sample data from the historical completed projects to determine the predicted value of the target project.

Example: We plan to develop an OA system to support E-government needs such as online office work for government departments.

Main attribute identification: The three main attributes of a project can be identified as development type, business domain, and application type, namely "new development", "government", and "OA".

13.1 Software Size Measurement

■ Software size estimation approach: Comparison(类比法)

Example: E-government software size estimation

Querying the industry benchmark database, the workload data (in person hours/PH) in the following table.

属 性	项 目 数 量	P10	P25	P50	P75	P90
新开发	105	1005	1983	5892	12406	98727
政府	52	892	2416	4713	9319	43658
OA	34	576	2025	5128	7144	21990

Software size estimation:

The most likely value of the required workload for this project is $(5892+4713+5128)/3$, which is 5244 person hours.

The reasonable range for workload estimation is roughly between 2141 person hours and 9623 person hours, using the values of P25 and P75 to calculate the average values respectively.

13.1 Software Size Measurement

■ Software size estimation approach: Analogy(类推法)

Analogy method is based on being able to find a completed project that is a very good match to your proposed project based on its major attributes. The project delivery rate and speed of delivery from the analog are then used to guide the estimate of the effort and duration for your new project.

The analogy method belongs to the method mainly based on "estimation". Compare and calculate the target project with one or more past projects, identify differences that are particularly similar or/and differences, and finally adjust the actual software size based on these differences.

When using the analogy method, it should be noted that the selected historical project and the project to be evaluated must be highly similar. Historical data should be selected from within the organization as much as possible, and adjustments must be made for differences.

13.1 Software Size Measurement

■ Software size estimation approach: Analogy(类推法)

Example:

Project Description: Government Department A plans to develop a new OA system to support e-government needs.

Historical project: Government department B has developed a similar system, but there are differences in functional requirements between departments A and B, but the differences in scale, difficulty, and quality requirements between the two projects are not significant.

Reference project: The total development period is 4.92 months, and the total workload is 4625 person hours. Including the project planning stage 78 person hours, the demand stage 555 person hours, the design stage 694 person hours, the construction stage 1619 person hours, the testing stage 922 person hours, and the handover stage 757 person hours, $78+555+694+1619+922+757=4625$ person hours.

Size estimation: Considering that the project can use the system developed for Department B as a prototype to understand customer requirements, assuming that the requirements analysis phase can reduce approximately one-third of the workload, the estimated project size is $555 \times 2/3 + 694 + 1619 + 922 + 757 = 4440$ person hours.

13.1 Software Size Measurement

■ Producing a detailed estimation

Attributes for Estimation by Comparison and Analogy.

Project type	Development, Enhancement, or Redevelopment (on a new platform).
Size	Functional size measurement.
Project goals	In terms of quality, cost, schedule, and constraints (that is, priority of each). Note that <i>cost</i> , <i>scope</i> (functionality and quality), and <i>time</i> (effort) are the famous “triple constraint” of project management.
Development platform	Mainframe, midrange, PC, or multiplatform.
Language	Programming language or language level.
Task selection	Similar project profile in terms of activities and deliverables from those activities. (Phases and work activities included.)

13.1 Software Size Measurement

■ Software size estimation approach: Work Breakdown

In this method, the effort and duration associated with each component or activity of the software project is separately estimated and the results aggregated to produce an estimate of the whole job. This is a bottom-up technique.

Work breakdown method is useful when the project scope is well defined and an accurate work breakdown structure can be defined.

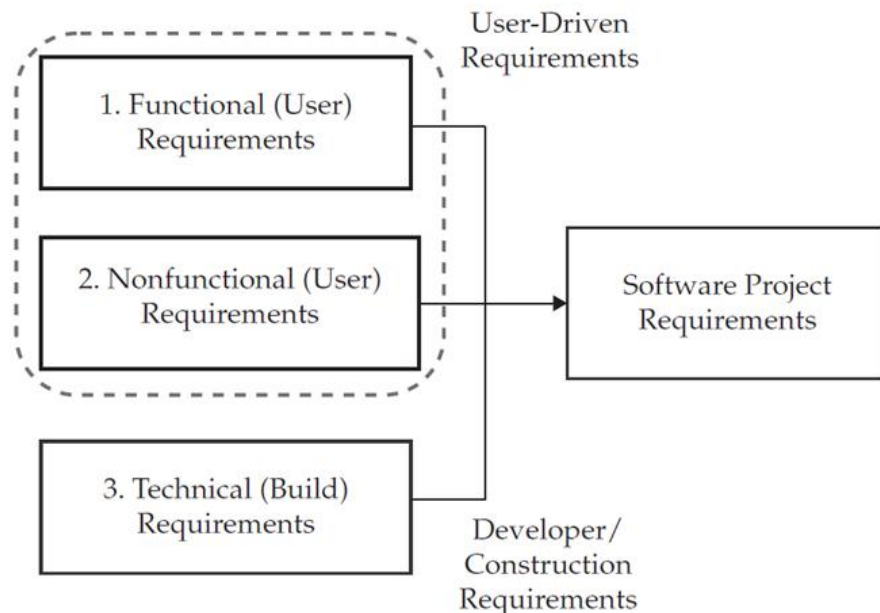
Typically, experienced project team members estimate their project tasks based on historical completed similar tasks, and the overall estimate is the aggregated sum of all work breakdown structure task estimates.

13.1 Software Size Measurement

■ Software size estimation approach: Equation(方程法)

In equation method, the size of the project is applied to an appropriate equation that has been derived from data of finished real projects. At present, the equation method are the mainstream methods for evaluating software size.

To make sense of functional size measurement and where it fits with estimating, it is useful to discuss the three types of software project requirements, Functional requirements, Nonfunctional requirements and Technical requirements.



13.1 Software Size Measurement

■ Functional size: a typical equation method

Functional size represents the size of the functional requirements.

Functional size is an important input in software cost estimation, but it is only one of a number of required variables.

For a new development project, functional size is the size of all of the delivered or installed functionality (analogous to a building's floor plan).

For an enhancement project, functional size is the total size of all functional requirements that are new, renovated (changed), or removed (deleted) from the software.

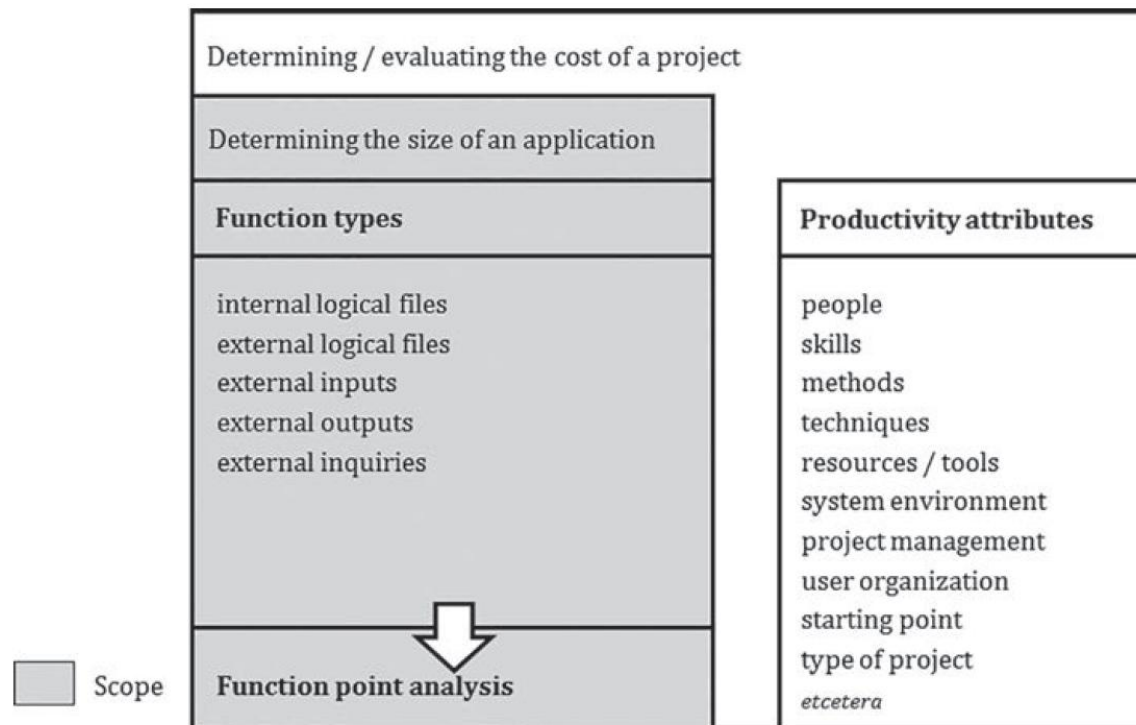
Nonfunctional requirements fall outside functional size. The indicator, value adjustment factor (VAF), which is an optional step in the IFPUG (International Function Point Users Group) function point method, is intended to address a portion of nonfunctional requirements.

13.1 Software Size Measurement

■ Scope

The Function Point Measurement(FPM) focuses on how the functional size of an application is determined.

FPA does not go into any of the aspects that play a role when project budgets are established on the basis of this functional size, e.g. productivity standards and productivity attributes. It only focuses on the application itself.



13.1 Software Size Measurement

■ Functional requirements

Functional requirements, represent **WHAT** functions will be included in the software. They are the business processes performed by or supported by the software, for example, record and store ambient temperature.

Functional requirements include the functions that the software must perform. The size of functional requirements is expressed in function points (FPs).

A function point(FP) is a unit of measure for functional size as defined within the IFPUG Functional Size Measurement Method.

13.1 Software Size Measurement

■ Nonfunctional requirements

Nonfunctional requirement, is the second type of software requirement and represents **HOW** the software must perform.

Nonfunctional requirements describe how the software must operate and are not included in functional size. Sometimes known as “quality requirements,” the nonfunctional requirements include suitability, accuracy, interoperability, compliance, security, reliability, efficiency, maintainability, portability, and quality in use, as described by the ISO standard ISO/IEC 9126 series, plus a range of performance requirements.

The nonfunctional requirements are the contracted specifications for the software product and include requirements for security (e.g. data encryption), performance (e.g. response time and reliability), accuracy (e.g. governmental approvals required), and other specifications of how the software must perform.

13.1 Software Size Measurement

■ Technical requirements

The technical requirements include hardware and software requirements, infrastructure requirements, database type, and so on.

These requirements address how the software will be developed or “built” and include tools, methods, type of project, resource skill levels, and also include where architectural design, configuration management methods, development methodology, use of packages, and use of CASE (Computer Aided Software Engineering) tools come into play.

Although technical requirements will not affect the measurement of software functional points, they will have an important impact on productivity and software cost estimation.

13.1 Software Size Measurement

■ Standards of software size/point estimation

Application functional size is measured by using the following functional size measurement standards such as

- COSMIC (Common Software Measurement International Consortium)
- FiSMA (Finnish Software Measurement Association)
- IFPUG (International Function Point User Group)
- NESMA (Netherlands Software Metrics Association)
- Mark II Function Point Analysis (U.K)
- **Chinese National Standard** (GB/T 36964-2018)

The standards marked in blue are the main standard used in software industry in China currently.

ICS 35.080
L 77



中华人民共和国国家标准

GB/T 36964—2018

软件工程 软件开发成本度量规范

Software engineering—Specification for software development cost measurement

2018-12-28 发布

2019-07-01 实施

国家市场监督管理总局
中国国家标准化管理委员会 发布

13.1 Software Size Measurement

■ Types of Counts

Functional size is a size of the software derived by quantifying the Functional User Requirements. Functional User Requirements (FURs) are a subset of the user requirements specifying what the software should do in terms of tasks and services.

Functional User Requirements include but are not limited to:

- Data transfer — for example, adding a new order, sending an invoice, forwarding location coordinates.
- Data transformation — for example, calculating the cost of that order to send an invoice, determining the location coordinates.
- Data storage — for example, storing the new order, saving location information.
- Data retrieval — for example, search and display order information, search and display location information.

Function point counts can be determined, based on the purpose, for either projects or applications, as one of the following:

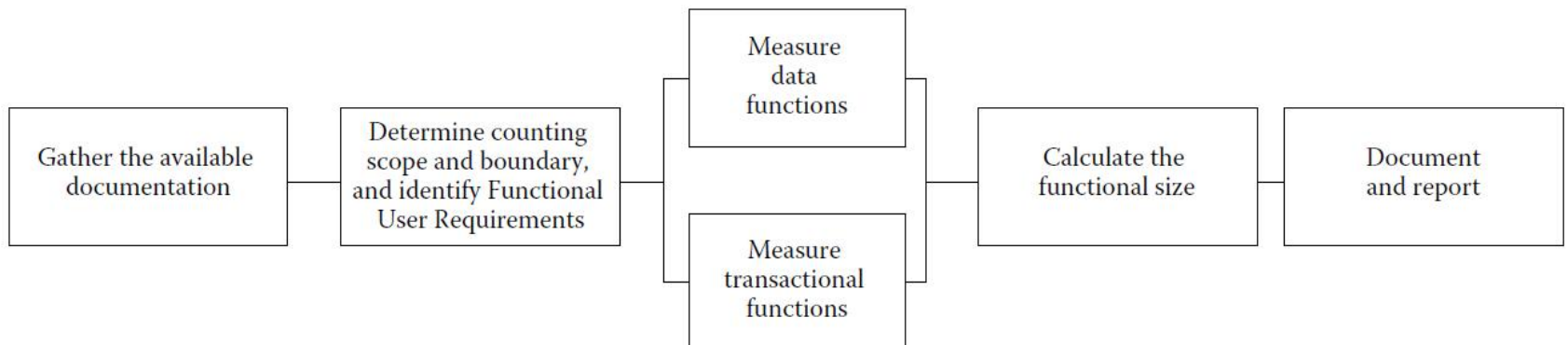
- Development project function point count.
- Enhancement project function point count.
- Application function point count.

13.1 Software Size Measurement

■ Function Point Counting Procedure

The following are the steps in the function point counting procedure:

- Gather the available documentation.
- Determine counting scope and boundary, and identify Functional User Requirements.
- Measure data functions.
- Measure transactional functions.
- Calculate the functional size.
- Document and report.



13.1 Software Size Measurement

■ Function Point Counting Procedure

The functional size can be measured for either projects or applications.

A development project function point count is the activity of applying the Functional Size Measurement (FSM) Method to measure the functional size of a development project. It includes all functions impacted (*built* or *customized*) by the project activities. It also includes functions developed as part of the development project.

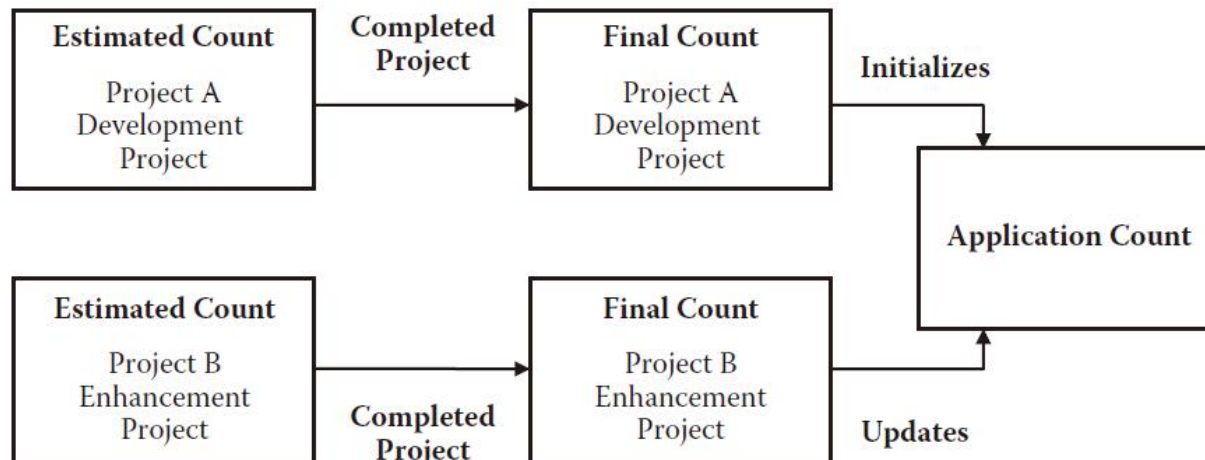
A development project function point count must often be updated as development proceeds. These subsequent counts would not start from scratch, but they would validate previously identified functionality and attempt to capture added functionality, commonly called “scope creep.” Counts often occur throughout the development process.

13.1 Software Size Measurement

■ Function Point Counting Procedure

An enhancement project function point count is the activity of applying the Functional Size Measurement (FSM) Method to measure the functional size of an enhancement project. It includes all the functions being added, changed, and deleted. It also includes conversion functions developed as part of the enhancement project.

A development project is often estimated a number of times before its completion (see following Figure). At the time of completion, a final development project count is measured, and an application count is determined. Later, enhancements to that application can be estimated until their completion. Upon completion of each enhancement project, a final enhancement project count is measured, and the application count is modified to reflect the changes.



13.1 Software Size Measurement

■ Use of FPM: application vs project

Functional size of a new development applications

This pertains to the functional size of applications in the process of being built or that have already been built at the request of a user or user organization, and that provide a solution to the needs or wishes of the user or user organization.

If the application in a development project is realized in a single project, determining the functional size of an application does not occur differently than when determining the functional size of a project. Notice, however, that the size of any conversion software shall not be counted when the functional size of an application is determined.

If the application is being realized in the form of a number of sub-projects carried out in parallel, then the total functionality furnished by all the sub-projects will have to be examined in order to determine the functional size of an application. When examining these sub-projects, make sure that functionality appearing in more than one sub-project (such as a logical file) is not counted twice.

13.1 Software Size Measurement

■ Use of FPM: application vs project

Functional size of enhanced applications

Step1: Determine the number of function points of the application before the change (**AFPB**).

Step2: Identify which transactions and/or logical files are added to the application and establish how many function points they represent (**ADD**).

Step3: Determine which transactions and/or logical files are deleted from the existing application and count how many function points they represent (**DEL**).

Step4: Establish which transactions and/or logical files change. Then determine the number of function points that they represent before the change (**CHGB**) and after the change (**CHGA**).

Step5: Determine the functional size of the application after the enhancement (**AFPA**) as follows:

$$\text{AFPA} = [\text{AFPB} + \text{ADD} - \text{DEL} + (\text{CHGA} - \text{CHGB})]$$

13.1 Software Size Measurement

■ Use of FPM: application vs project

Functional size of enhancement projects

An enhancement project considers enhancements to one or more existing applications. This means that functionality can be added to, changed in, and deleted from these applications. The steps required to determine the functional size of the project in function points are as follows.

Step1: Identify which transactions and/or logical files are going to be added to the application(s) and establish how many function points they represent (**ADD**).

Step2: Determine which transactions and/or logical files are going to be deleted from the existing application(s) and determine how many function points they represent (**DEL**).

Step3: Establish which transactions and/or logical files change. Then determine the number of function points they represent after the change (**CHGA**).

Step4: Calculate the functional size for the enhancement project as follows (**EFP**):

$$\text{EFP} = \text{ADD} + \text{DEL} + \text{CHGA}$$

13.1 Software Size Measurement

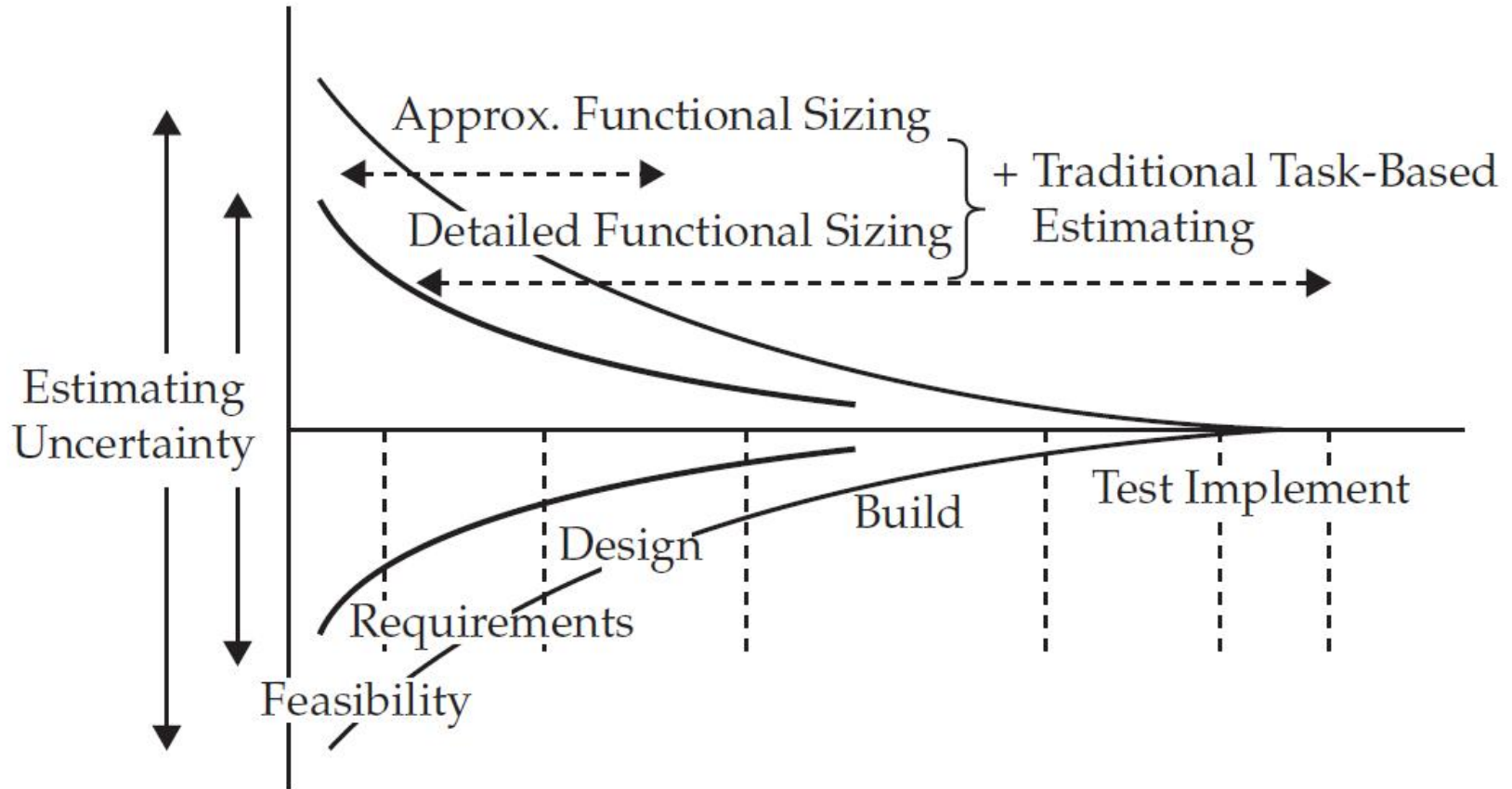
■ Use of FPM: Application vs Project

Table13.1 Functional Size of a project versus an application

Size type	Lifecycle stage				
	Initial release	Release 1		Release 2	
ADD	1000	200		500	
DELETE		40		100	
CHANGE	Before	80		220	
	After	100		200	
Project size	1000	ADD	200	ADD	500
		CHANGE After	100	CHANGE After	200
		DELETE	40	DELETE	100
		TOTAL	340	TOTAL	800
Application size	1000	Initial release	1000	Release 1	1180
		ADD	+200	ADD	+500
		CHANGE After	+100	CHANGE After	+200
		CHANGE Before	-80	CHANGE Before	-220
		DELETE	-40	DELETE	-100
		TOTAL	1180	TOTAL	1560

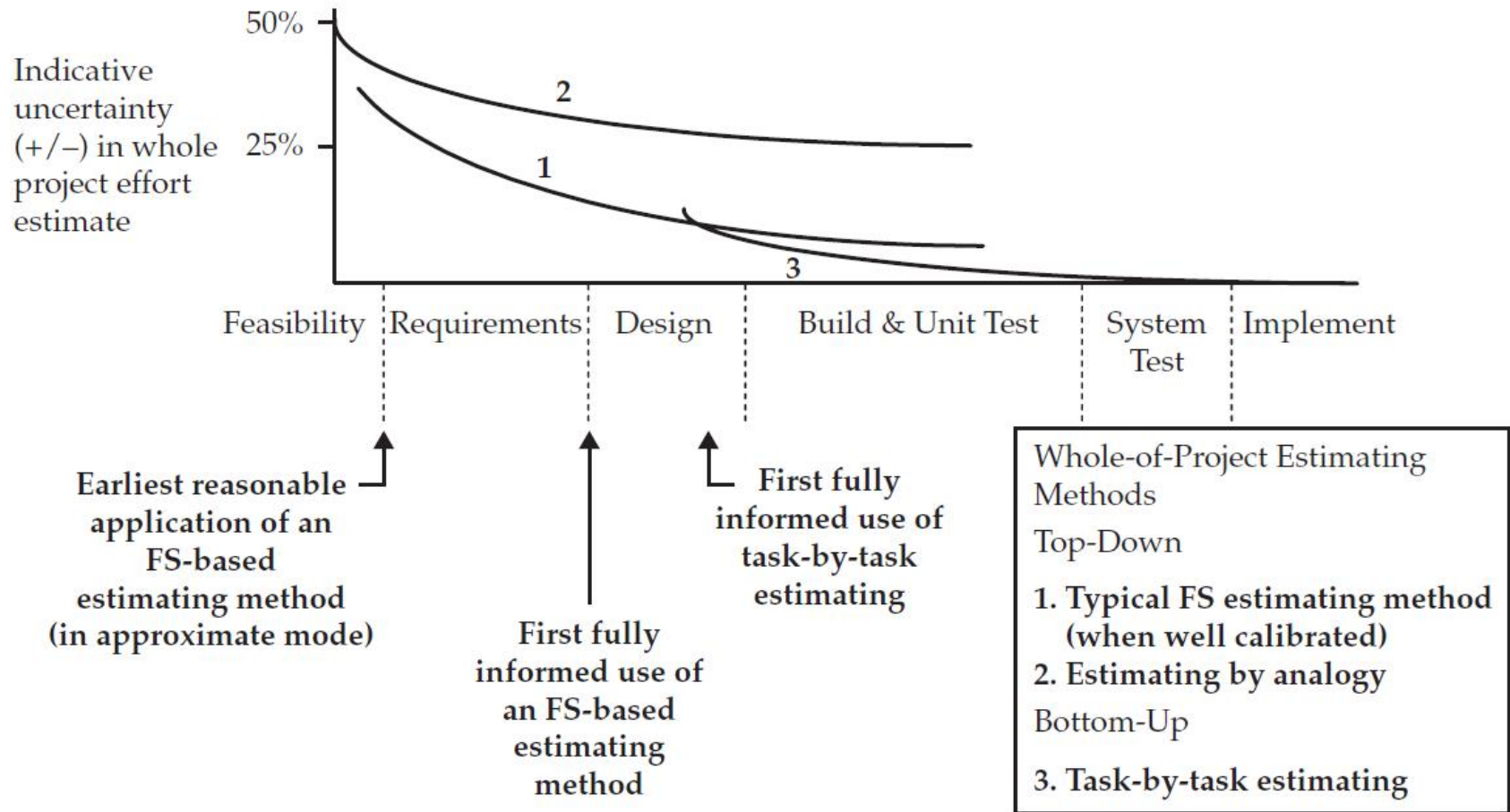
13.1 Software Size Measurement

■ The cone of uncertainty of estimation



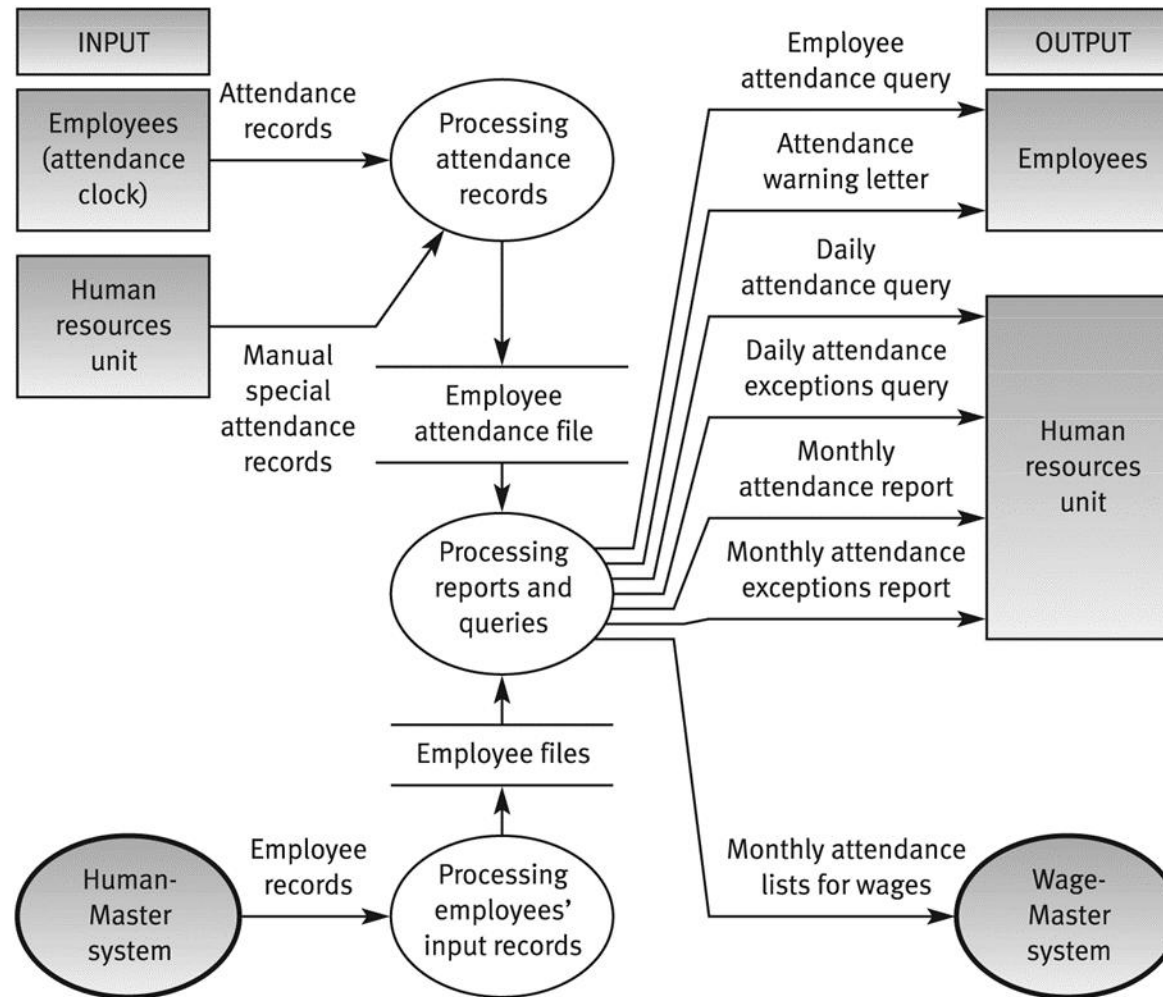
13.1 Software Size Measurement

■ Estimating methods in development phase



13.1 Software Size Measurement

■ Case study



13.1 Software Size Measurement

■ Case study

Calculation of CFP Crude Function Points

Analysis of the software system as presented in the DFD summarizes the number of the various components:

- Number of user inputs = 2
- Number of user outputs = 3
- Number of user online queries = 3
- Number of logical files = 2
- Number of external interfaces = 2

The degree of complexity (simple, average or complex) was evaluated for each component.

13.1 Software Size Measurement

■ Case study

The ATTEND MASTER - CFP calculation form

Software system components	Complexity level									Total CFP
	Simple			average			complex			
	Count	Weight Factor	Points	Count	Weight Factor	Points	Count	Weight Factor	Points	
	A	B	C= AxB	D	E	F= DxE	G	H	I= GxH	
User inputs	1	3	3	---	4	---	1	6	6	9
User outputs	---	4	---	2	5	10	1	7	7	17
User online queries	1	3	3	1	4	4	1	6	6	13
Logical files	1	7	7	---	10	---	1	15	15	22
External interfaces	---	5	---	---	7	---	2	10	20	20
Total CFP										81

13.1 Software Size Measurement

■ Case study

The ATTEND MASTER - RCAF calculation form

No	Subject	Grade
1	Requirement for reliable backup and recovery	0 1 2 3 4 5
2	Requirement for data communication	0 1 2 3 4 5
3	Extent of distributed processing	0 1 2 3 4 5
4	Performance requirements	0 1 2 3 4 5
5	Expected operational environment	0 1 2 3 4 5
6	Extent of online data entries	0 1 2 3 4 5
7	Extent of multi-screen or multi-operation online data input	0 1 2 3 4 5
8	Extent of online updating of master files	0 1 2 3 4 5
9	Extent of complex inputs, outputs, online queries and files	0 1 2 3 4 5
10	Extent of complex data processing	0 1 2 3 4 5
11	Extent that currently developed code can be designed for reuse	0 1 2 3 4 5
12	Extent of conversion and installation included in the design	0 1 2 3 4 5
13	Extent of multiple installations in an organization and variety of customer organizations	0 1 2 3 4 5
14	Extent of change and focus on ease of use	0 1 2 3 4 5
	Total = RCAF	41

13.1 Software Size Measurement

■ Case study

The ATTEND MASTER – Function points calculation

$$FP = CFP \times (0.65 + 0.01 \times RCAF)$$

$$FP = 81 \times (0.65 + 0.01 \times 41) = 85.86$$



Core Concepts

Methods of software size measurement

Function Point Analysis and its standards

Assignment

Learn Chapter 3 of Software Engineering Economics(软件工程经济学) MOOC. The linkage is as follow

<https://coursehome.zhihuishu.com/courseHome/1000050734#teachTeam>